



Università
Ca'Foscari
Venezia

Theorem Proving

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Soundness & Completeness

Want to prove theorem T from Axioms Ax

Chosen a set of inference rules R

- ▶ $A \models B$ means B is entailed by A
- ▶ $A \vdash B$ means B is derived from A using R

R should be **sound**: if $A \vdash B$ then $A \models B$

Want R to be **complete**: if $A \models B$ then $A \vdash B$

Soundness and completeness of R isn't enough

We want a reasonable search space

- ▶ R determines operators, so influences the space
- ▶ Can I see where I'm going? (Heuristic measure)
- ▶ Can I restrict options at each state? (Branching)

The Resolution Method

Proof by refutation with a single inference rule

- ▶ No need to worry about choice of rule
- ▶ Just how we apply the one rule

Resolution is complete for FOL

- ▶ Refutation-complete [Robinson 1965]
 - ▶ If $(\neg T, Ax)$ unsatisfiable it will derive a contradiction
- ▶ So it will prove any true theorem of FOL

Even so, it might take a long time ($>$ age of universe)

- ▶ Even fairly trivial theorems can take a long time
- ▶ Can use search heuristics to speed it up
- ▶ No guarantees if it's not a theorem

Binary Resolution

Unit resolution rule (last lecture)

$$\frac{A \vee B, \neg B}{A}$$

Binary resolution rule

$$\frac{A \vee B, \neg C \vee D}{\text{Subst}(\theta, A \vee D)}$$

if substitution θ is such that $\text{Subst}(\theta, B) = \text{Subst}(\theta, C)$

The literals B and $\neg C$ are resolved

- B and C have been made the same by θ

Resolution in FOL

What if KB contains non-disjuncts?

- ▶ Preprocessing step: **rewrite to Conjunctive Normal Form**

How do we find substitution θ ?

- ▶ The **unification** algorithm

But what if more than two disjuncts?

- ▶ Extend binary resolution to **full resolution**

Conjunctive Normal Form

A conjunction of **clauses**

- ▶ Each clause is a disjunction of literals
- ▶ **Prop. literal**: proposition or negated proposition
- ▶ **FO literal**: predicate or negated predicate
- ▶ No quantifiers (all variables implicitly universal)

Example FO clause

$\text{likes}(\text{george}, X) \vee \neg \text{likes}(\text{tony}, \text{house_of}(\text{george})) \vee \neg \text{is_mad}(\text{maggie})$

Conjunctive Normal Form

Any FO sentence can be rewritten as CNF

1. Eliminate implication/equivalence (rewrite)
2. Move $\neg$ inwards (rewrite)
3. Rename variables apart (substitution)
4. Move quantifiers outwards (rewrite)
5. Skolemise existential variables (substitution)
6. Distribute $\wedge$ over $\vee$ (rewrite)
7. Flatten binary connectives (rewrite)

Example CNF Conversion

Propositional example

$$(B \vee (A \wedge C)) \rightarrow (B \vee \neg A)$$

1. Remove implication:

$$\neg(B \vee (A \wedge C)) \vee (B \vee \neg A)$$

2. Move $\neg$ inwards (De Morgan's x 2):

$$\begin{aligned} &(\neg B \wedge \neg(A \wedge C)) \vee (B \vee \neg A) \\ &(\neg B \wedge (\neg A \vee \neg C)) \vee (B \vee \neg A) \end{aligned}$$

(Skip 3 to 5 as there are no variables.)

Example CNF Conversion

6. Distribute $\vee$ over $\wedge$:

$$(\neg B \vee (B \vee \neg A)) \wedge ((\neg A \vee \neg C) \vee (B \vee \neg A))$$

7. Flatten connectives

$$(\neg B \vee B \vee \neg A) \wedge (\neg A \vee \neg C \vee B \vee \neg A)$$

Drop 1st clause $(\neg B \vee B)$, remove duplicate $\neg A$ from 2nd:

$$\neg A \vee \neg C \vee B$$

Skolemisation

Replace $\exists V$ with a 'something' term

- ▶ If no preceding $\forall U$ use fresh Skolem constant
- ▶ Otherwise fresh Skolem **function** parameterized by all preceding universally closed variables

$$\forall X. \exists Y. (\text{person}(X) \rightarrow \text{has}(X, Y) \wedge \text{heart}(Y))$$

gets converted to

$$\text{person}(X) \rightarrow \text{has}(X, f(X)) \wedge \text{heart}(f(X))$$

(The particular heart $f(X)$ depends on person X)

Unification

Propositional inference rules used in first-order logic

But in FOL we can make substitutions

- ▶ Sometimes a substitution can allow a rule to be applied
- ▶ cf. FO binary resolution

$$\begin{array}{c} \text{knows}(\text{john}, X) \rightarrow \text{hates}(\text{john}, X) \\ \text{knows}(\text{john}, \text{mary}) \end{array}$$

Substitution + Modus Ponens: infer $\text{hates}(\text{john}, \text{mary})$

Need to find substitution that makes literals equal (unifier)

Unification

Want an algorithm which:

- ▶ Takes two FOL sentences as input
- ▶ Outputs a substitution X/mary , Y/Z , etc.
 - ▶ Which assigns terms to variables in the sentences
 - ▶ So that the first sentence looks exactly like the second
- ▶ Or fails if there is no way to unify the sentences

$$\text{Unify}(\text{knows}(\text{john}, X), \text{knows}(\text{john}, \text{mary})) = \{X/\text{mary}\}$$

$$\text{Unify}(\text{knows}(\text{john}, X), \text{knows}(\text{jack}, \text{mary})) = \text{Fail}$$

Unification

Example

```
def unify_internal(x, y, mu):  
    if (mu==failure): return failure  
    if (x==y): return mu  
    if (isa_variable(x)): return  
        unify_variable(x, y, mu)  
    if (isa_variable(y)): return  
        unify_variable(y, x, mu)  
    if (isa_compound(x) and isa_compound(y)):  
        return unify_internal(args(x), args(y),  
            unify_internal(op(x), op(y), mu))  
    if (isa_list(x) and isa_list(y)):  
        return unify_internal(x[1:], y[1:],  
            unify_internal(x[0], y[0], mu))  
    return failure
```

Unification

Example

```
def unify_variable(var, x, mu):  
    if (subst(var, val) in mu): return  
        unify_internal(val, x, mu)  
    if (subst(x, val) is in mu): return  
        unify_internal(var, val, mu)  
    if (occurs(var, x)): return failure  
    mu.append(subst(var, x))  
    return mu
```

The Occurs Check

Suppose we needed to substitute X with $f(X, Y)$

- ▶ This would give us $f(X, Y)$ instead of X
- ▶ But there is still an X in there, so the substitution isn't complete:
 - ▶ we need $f(f(X, Y), Y)$, then $f(f(f(X, Y), Y), Y)$ and so on

We need to avoid this situation

- ▶ Otherwise the algorithm won't stop
- ▶ Case 3 in `unify_variable` checks this
- ▶ Known as the “occurs check”

Occurs check slows the algorithm down

- ▶ Order $O(n^2)$, where n is size of expressions being unified

An Example Unification

Suppose we want to unify these sentences:

1. $p(X, \text{tony}) \wedge q(\text{george}, X, Z)$
2. $p(f(\text{tony}), \text{tony}) \wedge q(B, C, \text{maggie})$

By inspection, this is a good substitution:

$$\{X/f(\text{tony}), B/\text{george}, C/f(\text{tony}), Z/\text{maggie}\}$$

This makes both sentences become:

$$p(f(\text{tony}), \text{tony}) \wedge q(\text{george}, f(\text{tony}), \text{maggie})$$

Note that we want to substitute X for C

- But we have substituted $f(\text{tony})$ for X already

Requires five iterations of the unification algorithm

Resolution Revisited

Binary resolution rule (using unification)

$$\frac{A \vee B, \neg C \vee D}{\text{Subst}(\theta, A \vee D)} \quad \text{if } \text{Unify}(B, C) = \theta$$

Unification algorithm finds Most General Unifier (MGU) θ

- Don't substitute any more than need to

Resolution Revisited

Full resolution

If $\text{Unify}(P_j, \neg Q_k) = \theta$ ($\neg$ makes them unifiable)

$$\frac{P_1 \vee \dots \vee P_m, Q_1 \vee \dots \vee Q_n}{\text{Subst}(\theta, P_1 \vee \dots (no P_j) \dots \vee P_m \vee Q_1 \vee \dots (no Q_k) \dots \vee Q_n)}$$

P_j and Q_k are resolved

- ▶ Arbitrary number of disjuncts
- ▶ Relies on preprocessing into CNF

Resolution Proving

Knowledge base of clauses

- ▶ Start with the axioms and negation of theorem in CNF

Resolve pairs of clauses

- ▶ Using single rule of inference (full resolution)
- ▶ Resolved sentence contains fewer literals

Resolution Proving

Proof ends with the empty clause

- ▶ Signifies a contradiction
- ▶ Must mean the negated theorem is false
 - ▶ (Because the axioms are consistent)
- ▶ Therefore the original theorem was true
- ▶ This can only be because:
 - ▶ Two unit clauses were resolved
 - ▶ One was the negation of the other (after substitution)
Example: $q(X)$ and $\neg q(X)$ or $p(X)$ and $\neg p(bob)$
- ▶ Hence if we see the empty clause
 - ▶ This was because there was an inconsistency
 - ▶ Hence the proof by refutation

Resolution as Search

Initial State: Knowledge base (KB) of axioms and negated theorem in CNF

Operators: Resolution rule picks 2 clauses and adds new clause

Goal Test: Does KB contain the empty clause?

Search space of KB states

We want proof (path) or just checking (artefact)

Example

Socrates is a man and all men are mortal Therefore Socrates is mortal

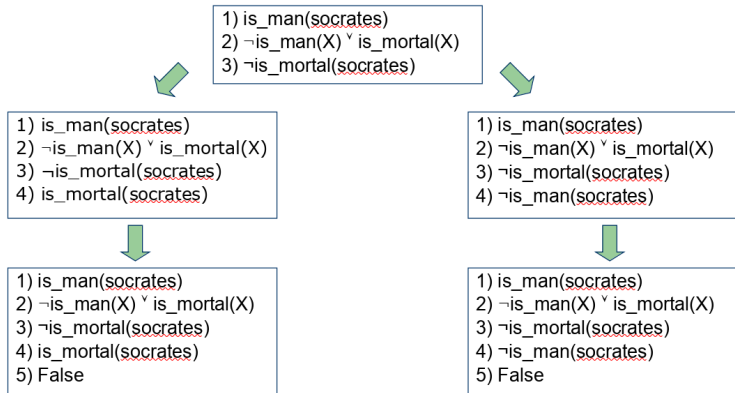
Initial state

1. $\text{is_man}(\text{socrates})$
2. $\neg \text{is_man}(X) \vee \text{is_mortal}(X)$
3. $\neg \text{is_mortal}(\text{socrates})$ (negation of theorem)

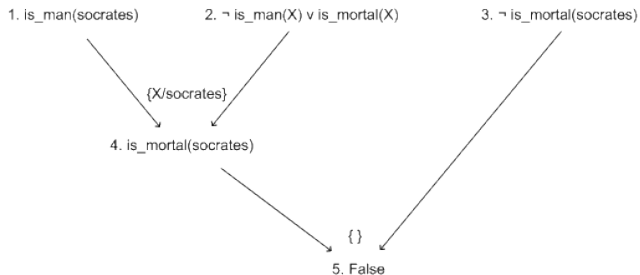
Resolving (1) & (2) gives new state

4. $\text{is_mortal}(\text{socrates})$

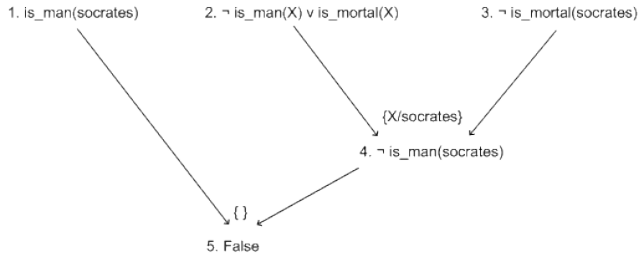
Example



Resolution Proof Tree 1



Resolution Proof Tree 2



You said that all men were mortal.

If we assume that Socrates is not mortal, then, given your previous statement, this means Socrates is not a man [first resolution step].

But you said that Socrates is a man, which means that our assumption was false [second resolution step].

So Socrates must be mortal.

Heuristic Strategies

Pure resolution search tends to be slow

For interesting problems

- ▶ Many clauses in the initial knowledge base
- ▶ Each step adds a new clause (which can be used)
- ▶ Num. of possible resolution combinations explodes

Selection Heuristics

- ▶ Intelligently choose which pair to resolve

Pruning Heuristics

- ▶ Forbid certain pairs

Unit Preference Strategy

Prefer to resolve unit clauses (that contain only a single literal)

Searching for smallest (empty) clause

- ▶ Resolving with the unit clauses keeps small

Very effective early on for simple problems

- ▶ Doesn't reduce branching rate for medium problems

Set of Support Strategy

Distinguished subset of KB clauses

- ▶ Set of support (SOS) clauses
- ▶ Every step must involve SOS (pruning heuristic)

Must be careful not to lose completeness

Example SOS strategy:

- ▶ Initial SOS is negated theorem
- ▶ Add new clauses to SOS
- ▶ Hence False will be deduced (strategy is complete)

Many provers use SOS, e.g. Prover9

Subsumption

Clause C **subsumes** clause D if C is more 'general' (D is more specific)

Naive check for subsumption

- ▶ Select C_2 , a subset of literals of C
- ▶ Find $\theta = \text{Unify}(C_2, D)$
- ▶ θ does not add anything to D (only renames vars)

Example:

- ▶ $p(\text{george}) \vee q(X)$ subsumed by $p(A) \vee q(B) \vee r(C)$
- ▶ Substitution: $\{A/\text{george}, X/B\}$
- ▶ Second clause is more general

Subsumption Strategy

Check each new clause is not subsumed by KB

Complete strategy

- ▶ Specific clauses can be inferred from general ones
- ▶ So we can throw specific clauses away
- ▶ Reduced search space still contains False

Can be inefficient

- ▶ expense must be outweighed by the reduction in the search space